

UNIVERSIDAD ANTONIO JOSE DE SUCRE

Programa de Ingeniería de Sistemas

ENTREGA 2 — Modelado de BD + Backend Inicial

Sistema de Gestión de Tutorías Universitarias

DER final normalizado (3FN) y Diccionario de Datos

Curso: Electiva Profesional II

Docente: Ing. Jaider Javier de la Rosa Bertel

Integrantes: Wadid Rivas Payares

Azael Espitia Toribio

Jhon Jader Bertel Ortega

Eduardo Julio Betin Avilez

Enlace repositorio [https://github.com/WRivxs/SGTU-Tutorias-](https://github.com/WRivxs/SGTU-Tutorias-Universitarias)

GitHub: Universitarias

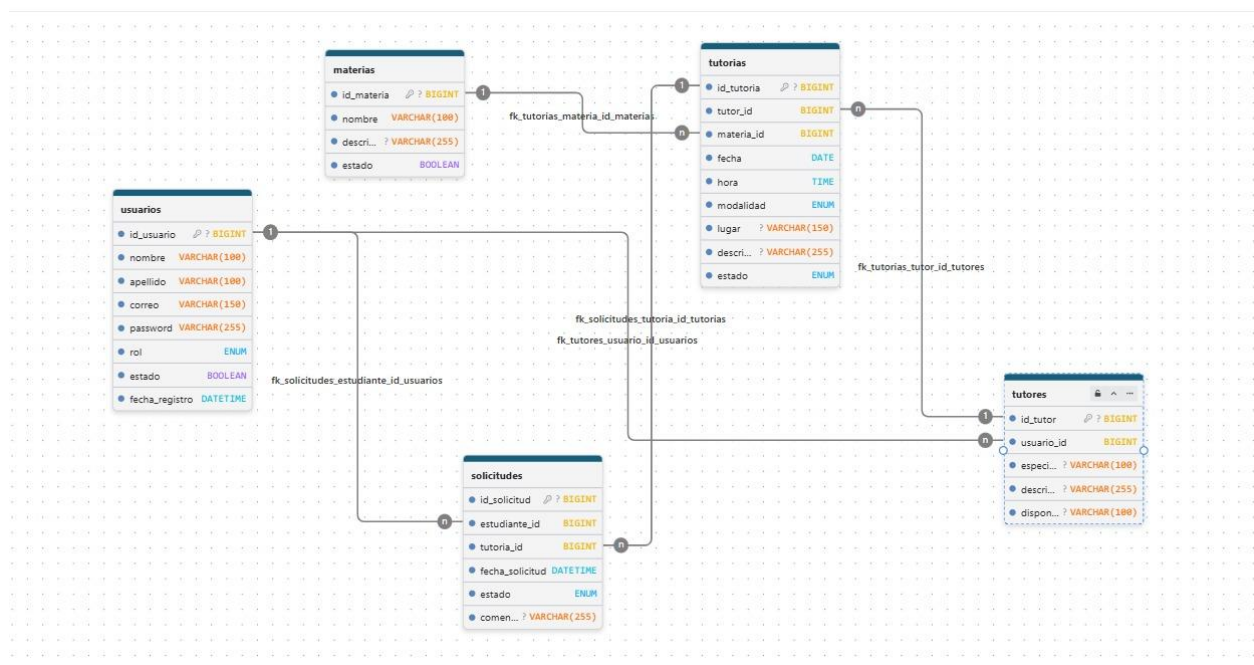
Fecha de entrega: 26/09/2026

1. Introducción a la Entrega

Esta entrega tuvo como objetivo consolidar el modelado de datos para el Sistema de Gestión de Tutorías Universitarias —evolucionando desde el diseño preliminar de la primera fase hacia un modelo relacional normalizado en 3FN (compuesto por cinco tablas: usuarios, asignaturas, tutores, sesiones de tutoría y solicitudes)— y desplegarlo en una instancia activa de AWS RDS. Sobre esta base, se implementó un backend inicial utilizando Spring Boot con una arquitectura en capas, verificándose las primeras operaciones CRUD del sistema mediante Postman.

2. Diagrama Entidad-Relación (DER) final

El siguiente diagrama representa el modelo relacional final de la base de datos del sistema, derivado del modelo preliminar presentado en la Entrega 1 y refinado para cumplir la Tercera Forma



Normal (3FN).

2. Justificación de la normalización (3FN)

2.1. Primera Forma Normal (1FN)

Todas las tablas del modelo almacenan valores atómicos: cada campo contiene un único dato indivisible (por ejemplo, nombre y apellido están separados, y no existen listas ni valores repetidos dentro de una misma columna). No hay grupos repetitivos ni columnas multivaluadas, por lo que el modelo cumple 1FN.

2.2. Segunda Forma Normal (2FN)

Todas las claves primarias del modelo son simples (de un solo atributo: `id_usuario`, `id_materia`, `id_tutor`, `id_tutoria`, `id_solicitud`). Al no existir claves primarias compuestas, no puede presentarse dependencia parcial: cada atributo no clave depende completamente de la clave primaria de su tabla. Por lo tanto, el modelo cumple 2FN.

2.3. Tercera Forma Normal (3FN)

Se verificó que ningún atributo no clave depende de otro atributo no clave (dependencia transitiva). Por ejemplo, en la tabla tutorias, el atributo 'lugar' depende únicamente de `id_tutoria` y no de `tutor_id` o `materia_id`. Los datos del tutor (especialidad, disponibilidad) se separaron en la tabla tutores en lugar de repetirse en tutorias, y los datos del usuario se separaron de los del tutor mediante la relación 1:0..1, evitando redundancia. De esta manera, el modelo cumple 3FN.

3. Diccionario de datos

Tabla: usuarios

Campo	Tipo de dato	Descripción	Restricciones
id_usuario	BIGINT	Identificador único del usuario	PK, AUTO_INCREMENT
nombre	VARCHAR(100)	Nombre del usuario	NOT NULL
apellido	VARCHAR(100)	Apellido del usuario	NOT NULL
correo	VARCHAR(150)	Correo electrónico, usado para iniciar sesión	NOT NULL, UNIQUE
password	VARCHAR(255)	Contraseña cifrada (hash)	NOT NULL
rol	ENUM	Rol del usuario: ADMINISTRADOR, TUTOR, ESTUDIANTE	NOT NULL
estado	BOOLEAN	Indica si la cuenta está activa	NOT NULL, DEFAULT TRUE
fecha_registro	DATETIME	Fecha y hora de creación de la cuenta	NOT NULL, DEFAULT CURRENT_TIMESTAMP

Tabla: materias

Campo	Tipo de dato	Descripción	Restricciones
id_materia	BIGINT	Identificador único de la materia	PK, AUTO_INCREMENT
nombre	VARCHAR(100)	Nombre de la materia	NOT NULL
descripcion	VARCHAR(255)	Descripción general de la materia	Opcional

estado	BOOLEAN	Indica si la materia está activa/ofertada	NOT NULL, DEFAULT TRUE
--------	---------	---	---------------------------

Tabla: tutores

Campo	Tipo de dato	Descripción	Restricciones
id_tutor	BIGINT	Identificador único del perfil de tutor	PK, AUTO_INCREMENT
usuario_id	BIGINT	Usuario asociado al perfil de tutor	FK → usuarios(id_usuario), UNIQUE, NOT NULL
especialidad	VARCHAR(100)	Área de especialidad del tutor	Opcional
descripcion	VARCHAR(255)	Descripción del perfil del tutor	Opcional
disponibilidad	VARCHAR(100)	Franja horaria general de disponibilidad	Opcional

Tabla: tutorias

Campo	Tipo de dato	Descripción	Restricciones
id_tutoria	BIGINT	Identificador único de la tutoría	PK, AUTO_INCREMENT
tutor_id	BIGINT	Tutor que ofrece la tutoría	FK → tutores(id_tutor), NOT NULL
materia_id	BIGINT	Materia de la tutoría	FK → materias(id_materia), NOT

			NULL
fecha	DATE	Fecha programada de la tutoría	NOT NULL
hora	TIME	Hora programada de la tutoría	NOT NULL
modalidad	ENUM	PRESENCIAL o VIRTUAL	NOT NULL
lugar	VARCHAR(150)	Lugar físico o enlace virtual	Opcional
descripcion	VARCHAR(255)	Detalle de la tutoría	Opcional
estado	ENUM	DISPONIBLE, OCUPADA, CANCELADA, FINALIZADA	NOT NULL, DEFAULT DISPONIBLE

Tabla: solicitudes

Campo	Tipo de dato	Descripción	Restricciones
id_solicitud	BIGINT	Identificador único de la solicitud	PK, AUTO_INCREMENT
estudiante_id	BIGINT	Estudiante que realiza la solicitud	FK → usuarios(id_usuario), NOT NULL
tutoria_id	BIGINT	Tutoría solicitada	FK → tutorias(id_tutoria), NOT NULL
fecha_solicitud	DATETIME	Fecha y hora en que se registró la solicitud	NOT NULL, DEFAULT CURRENT_TIMESTAMP

estado	ENUM	PENDIENTE, ACEPTADA, RECHAZADA, CANCELADA	NOT NULL, DEFAULT PENDIENTE
comentario	VARCHAR(255)	Comentario del estudiante o del tutor	Opcional

Relaciones del modelo

usuarios (1) — tutores (0..1): un usuario con rol TUTOR puede tener un único perfil de tutor.

usuarios (1) — solicitudes (N): un usuario con rol ESTUDIANTE puede realizar varias solicitudes.

materias (1) — tutorías (N): una materia puede tener varias tutorías programadas.

tutores (1) — tutorías (N): un tutor puede ofrecer varias tutorías.

tutorías (1) — solicitudes (N): una tutoría puede recibir varias solicitudes de distintos estudiantes.

4. Scripts de Creación de Base de Datos (DDL)

 Instrucción: Incluya los scripts SQL de creación de tablas (CREATE TABLE) documentados con comentarios. Puede pegar el bloque completo o anexarlo como archivo .sql junto con este documento.

```
-- Sistema de Gestión de Tutorías Universitarias
-- Script DDL - Base de datos MySQL (AWS RDS)
-- Entrega 2 - Modelado de BD + Backend Inicial
-- =====

CREATE DATABASE IF NOT EXISTS tutorias_db
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

USE tutorias_db;

-- -----
-- Tabla: usuarios
-- -----
```

```
CREATE TABLE usuarios (
  id_usuario BIGINT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  apellido VARCHAR(100) NOT NULL,
  correo VARCHAR(150) NOT NULL,
  password VARCHAR(255) NOT NULL,
  rol ENUM('ADMINISTRADOR','TUTOR','ESTUDIANTE') NOT NULL,
  estado BOOLEAN NOT NULL DEFAULT TRUE,
  fecha_registro DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT uq_usuarios_correo UNIQUE (correo)
) ENGINE=InnoDB;
```

```
-- -----
-- Tabla: materias
-- -----
```

```
CREATE TABLE materias (
  id_materia BIGINT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  descripcion VARCHAR(255),
  estado BOOLEAN NOT NULL DEFAULT TRUE
) ENGINE=InnoDB;
```

```
-- -----
-- Tabla: tutores
-- -----
```

```
CREATE TABLE tutores (
  id_tutor BIGINT AUTO_INCREMENT PRIMARY KEY,
  usuario_id BIGINT NOT NULL,
  especialidad VARCHAR(100),
  descripcion VARCHAR(255),
  disponibilidad VARCHAR(100),
  CONSTRAINT uq_tutores_usuario UNIQUE (usuario_id),
  CONSTRAINT fk_tutores_usuario FOREIGN KEY (usuario_id)
    REFERENCES usuarios(id_usuario)
    ON DELETE CASCADE
    ON UPDATE CASCADE
) ENGINE=InnoDB;
```

```
-- -----
-- Tabla: tutorias
-- -----
```

```
CREATE TABLE tutorias (
  id_tutoria BIGINT AUTO_INCREMENT PRIMARY KEY,
  tutor_id BIGINT NOT NULL,
  materia_id BIGINT NOT NULL,
  fecha DATE NOT NULL,
  hora TIME NOT NULL,
  modalidad ENUM('PRESENCIAL','VIRTUAL') NOT NULL,
  lugar VARCHAR(150),
  descripcion VARCHAR(255),
```

```

estado      ENUM('DISPONIBLE','OCUPADA','CANCELADA','FINALIZADA')
            NOT NULL DEFAULT 'DISPONIBLE',
CONSTRAINT fk_tutorias_tutor FOREIGN KEY (tutor_id)
            REFERENCES tutores(id_tutor)
            ON DELETE CASCADE
            ON UPDATE CASCADE,
CONSTRAINT fk_tutorias_materia FOREIGN KEY (materia_id)
            REFERENCES materias(id_materia)
            ON DELETE RESTRICT
            ON UPDATE CASCADE
) ENGINE=InnoDB;

-- -----
-- Tabla: solicitudes
-- -----

CREATE TABLE solicitudes (
  id_solicitud  BIGINT AUTO_INCREMENT PRIMARY KEY,
  estudiante_id BIGINT      NOT NULL,
  tutoria_id    BIGINT      NOT NULL,
  fecha_solicitud DATETIME   NOT NULL DEFAULT CURRENT_TIMESTAMP,
  estado        ENUM('PENDIENTE','ACEPTADA','RECHAZADA','CANCELADA')
                NOT NULL DEFAULT 'PENDIENTE',
  comentario    VARCHAR(255),
  CONSTRAINT fk_solicitudes_estudiante FOREIGN KEY (estudiante_id)
            REFERENCES usuarios(id_usuario)
            ON DELETE CASCADE
            ON UPDATE CASCADE,
  CONSTRAINT fk_solicitudes_tutoria FOREIGN KEY (tutoria_id)
            REFERENCES tutorias(id_tutoria)
            ON DELETE CASCADE
            ON UPDATE CASCADE
) ENGINE=InnoDB;

-- -----
-- Índices adicionales recomendados
-- -----

CREATE INDEX idx_tutorias_estado  ON tutorias(estado);
CREATE INDEX idx_solicitudes_estado ON solicitudes(estado);
CREATE INDEX idx_usuarios_rol     ON usuarios(rol);

-- -----
-- Datos de prueba (opcional)
-- -----

INSERT INTO usuarios (nombre, apellido, correo, password, rol) VALUES
('Admin', 'Sistema', 'admin@tutorias.com', '$2a$10$examplehash', 'ADMINISTRADOR'),
('Wadid', 'Rivas', 'wadid.tutor@tutorias.com', '$2a$10$examplehash', 'TUTOR'),
('Eduardo', 'Bettin', 'eduardo.estudiante@tutorias.com', '$2a$10$examplehash', 'ESTUDIANTE');

INSERT INTO materias (nombre, descripcion) VALUES
('Bases de Datos', 'Modelado y administración de bases de datos relacionales'),

```

```
('Programación Web', 'Desarrollo de aplicaciones web con frameworks modernos');
```

```
INSERT INTO tutores (usuario_id, especialidad, descripcion, disponibilidad) VALUES  
(2, 'Bases de Datos', 'Tutor con experiencia en modelado relacional', 'Lunes y miércoles 3-5pm');
```

5. Evidencia de instancia activa

Parámetro	Valor
DB instance identifier	parcial1electiva
Motor y versión	/MySQL 8.4.9
Clase de instancia	db.t4g.micro
Región	us-east-2 (Ohio)
Nombre de la base de datos	tutorias_db
Endpoint (sin exponer usuario/contraseña)	parcial1electiva.cd40okomwyz0.us-east-2.rds.amazonaws.com

5.1 Instancia AWS RDS

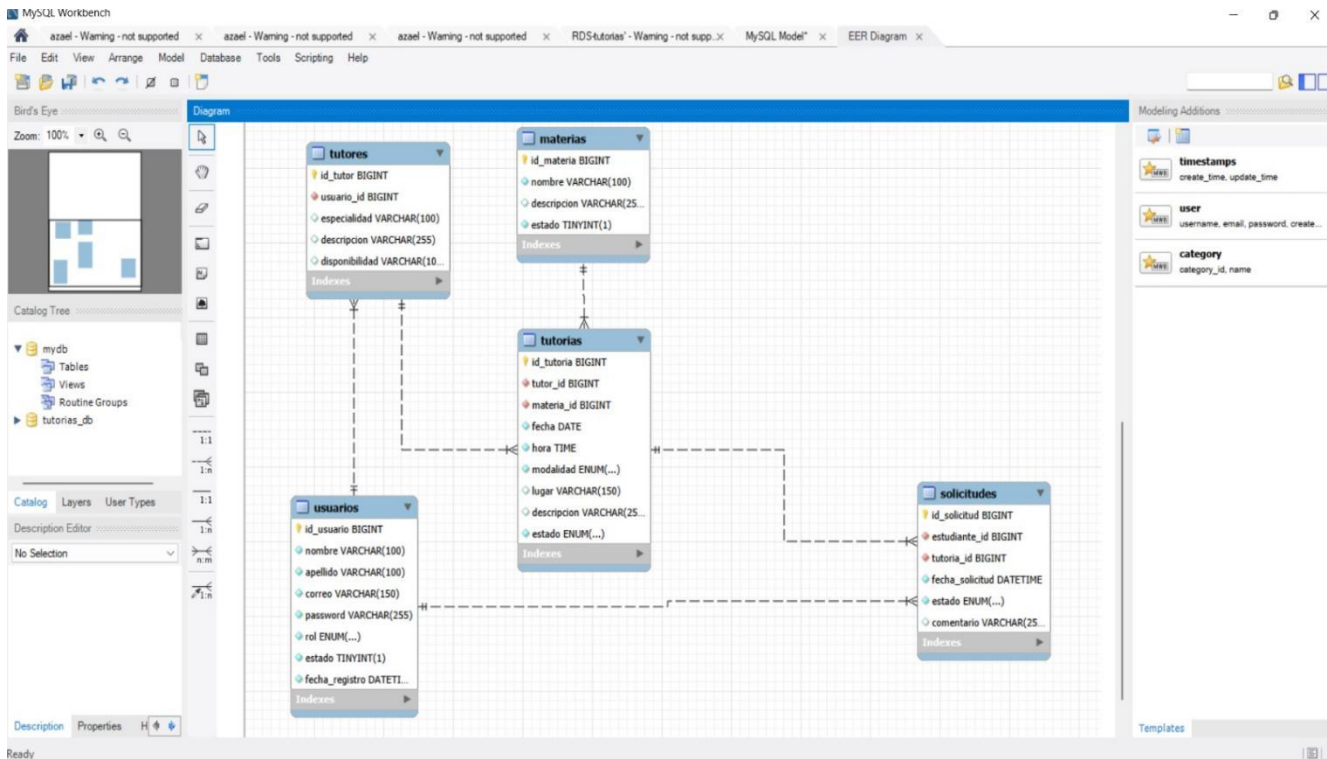
La instancia de MySQL se desplegó en Amazon RDS siguiendo la guía incluida en el repositorio (README.md). El script de creación de la base de datos (DDL) se encuentra en sql/ddl_tutorias.sql y fue ejecutado exitosamente sobre la instancia activa. La evidencia de la instancia activa (consola de AWS) y las capturas de Postman se anexan como archivos independientes en la entrega.

5.2 :

Gestión segura de credenciales

Las credenciales (DB_USERNAME, DB_PASSWORD) no están versionadas. Se declaran como variables de entorno en el script run-local.ps1 y se leen en src/main/resources/application.yml mediante la sintaxis \${DB_HOST} etc. En entornos de integración continua (GitHub Actions, GitLab CI) se utilizan Secrets para inyectar esas variables sin exponerlas.

5.3 Evidencia de Tablas Creadas en MySQL Workbench:



5. Arquitectura del Proyecto Spring Boot

6.1 Dependencias utilizadas

Dependencia	Propósito
spring-boot-starter-web	Exposición de endpoints REST (Tomcat embebido).
spring-boot-starter-data-jpa	Mapeo objeto-relacional y repositorios JPA.

Dependencia	Propósito
mysql-connector-j	Driver JDBC para MySQL/AWS RDS.
spring-boot-starter-validation	Bean Validation (@NotBlank, @Email, etc.).
lombok	Reducción de código boiler-plate (@Getter, @Setter, @Builder).
spring-boot-starter-test	JUnit 5 + Mockito para pruebas unitarias.

6.2 Estructura de paquetes

com.uaq.tutorias	
└─ config/	# Configuración global (CORS, Swagger, etc.)
└─ controller/	# @RestController - exposición HTTP
└─ service/	# Interfaces de lógica de negocio
└─ impl/	# Implementaciones (@Service, @Transactional)
└─ repository/	# Extiende JpaRepository para cada entidad
└─ entity/	# Clases JPA (@Entity) - modelo de BD
└─ exception/	# Manejo global de errores (@RestControllerAdvice)
└─ TutoriasBackendApplication.java	# Clase principal (main)

7. Entidades JPA Implementadas

 Instrucción: Documente al menos 3 entidades principales implementadas en esta entrega.

Entidad	Atributos principales	Anotaciones / relaciones clave
Usuario	id_usuario, nombre, apellido, correo, password, rol, estado, fecha_registro	@Entity, @Id @GeneratedValue(strategy = IDENTITY), @Enumerated(EnumType.STRING) en rol, @Column(unique = true) en correo
Tutor	/id_tutor, usuario, especialidad, descripcion, disponibilidad	@Entity, @OneToOne @JoinColumn(name = "usuario_id", unique = true) hacia Usuario

Entidad	Atributos principales	Anotaciones / relaciones clave
Tutoria	id_tutoria, tutor, materia, fecha, hora, modalidad, lugar, descripcion, estado	@Entity, @ManyToOne @JoinColumn(name = "tutor_id") hacia Tutor, @ManyToOne @JoinColumn(name = "materia_id") hacia Materia, @Enumerated(EnumType.STRING) en modalidad y estado

8. Repositorios (Spring Data JPA)

Repositorio	Entidad asociada	Métodos personalizados
UsuarioRepository	Usuario	findByCorreo(String correo), findByRol(RolUsuario rol)
TutorRepository	Tutor	findByUsuarioIdUsuario(Long idUsuario)
MateriaRepository	Materia	findByEstadoTrue()
TutoriaRepository	Tutoria	findByEstado(EstadoTutoria estado), findByTutorIdTutor(Long idTutor), findByMateriaIdMateria(Long idMateria)

9. Endpoints REST Implementados (CRUD)

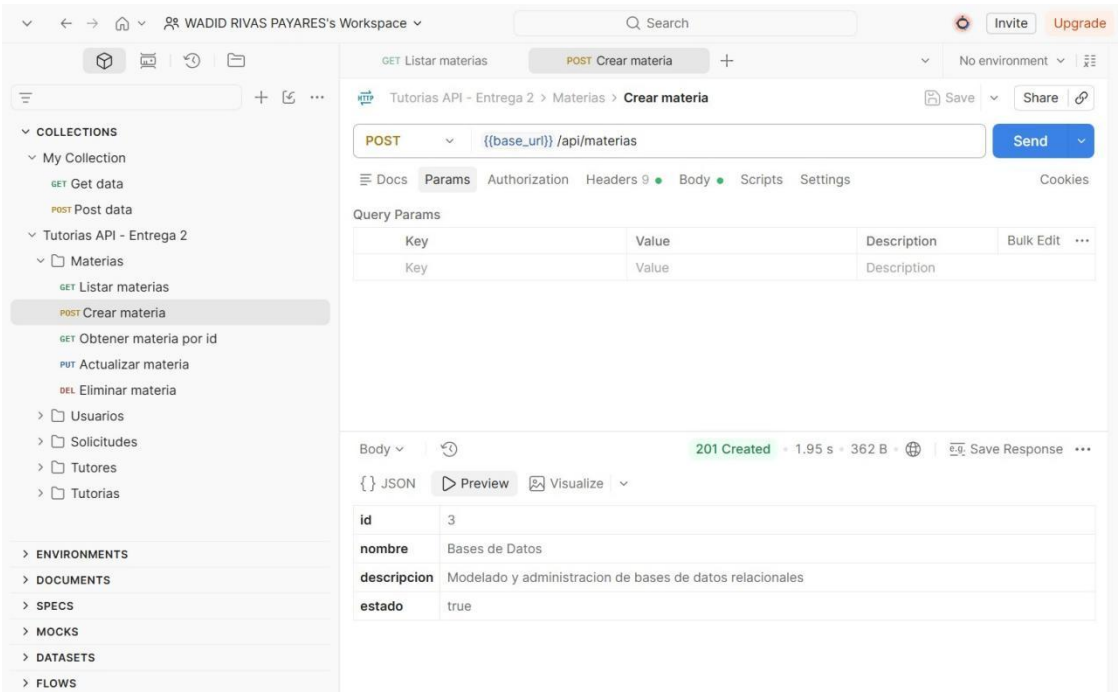
 Instrucción: Documente todos los endpoints CRUD implementados para al menos 2 entidades en esta entrega.

Método	Endpoint	Descripción	Entidad relacionada
GET	/api/materias	/Lista todas las materias registradas	Materia
GET /{id}	/api/materias/{id}	Obtiene una materia específica por su id	Materia
POST	/api/materias	Crea una nueva materia	Materia
PUT /{id}	/api/materias/{id}	Actualiza nombre, descripción o estado	Materia

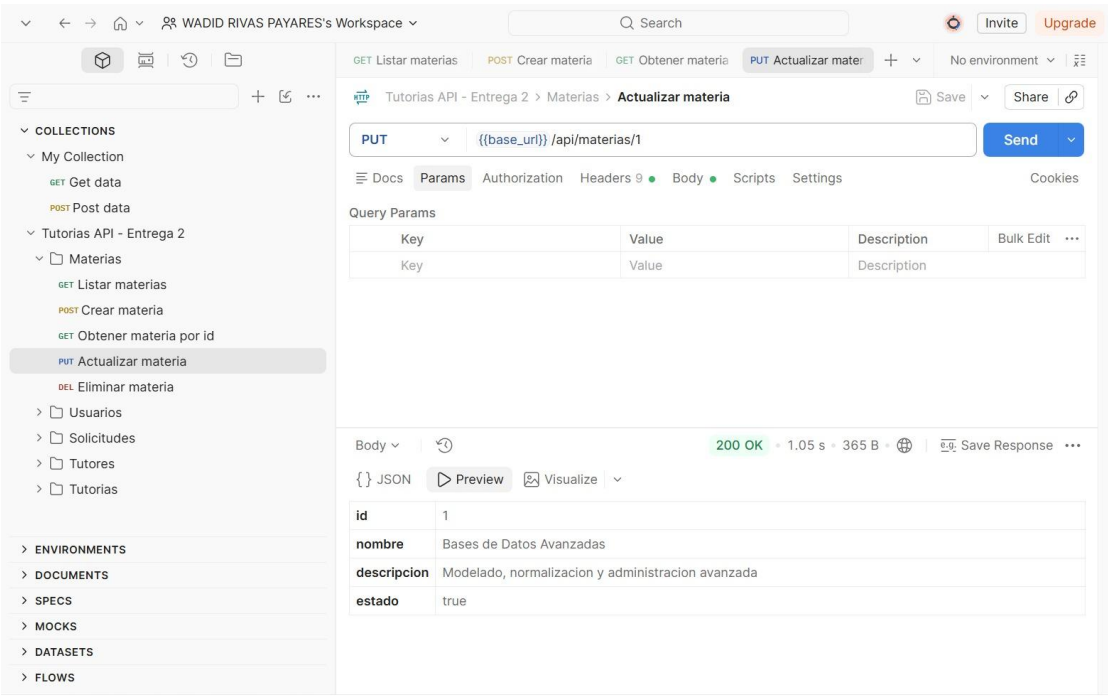
Método	Endpoint	Descripción	Entidad relacionada
		de una materia	
DELETE /{id}	/api/materias/{id}	Elimina una materia por su id	Materia
GET	/api/usuarios	Lista todos los usuarios registrados	Usuario
POST	/api/usuarios	[Registra un nuevo usuario	Usuario

10. Evidencia de Pruebas en Postman

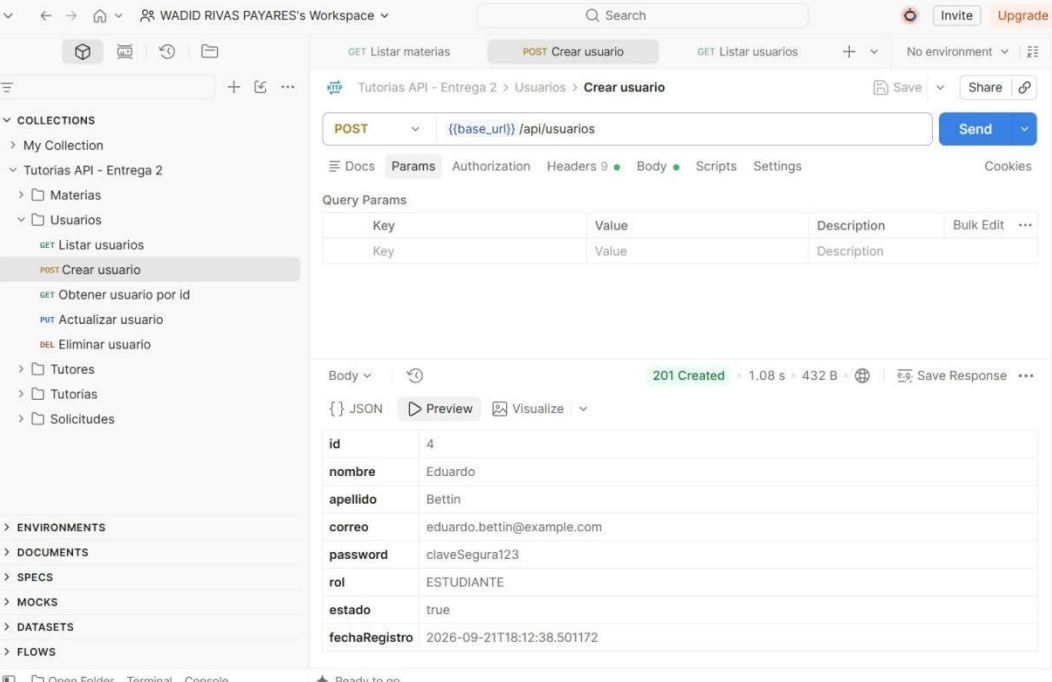
. Evidencias de Pruebas CRUD en Postman:



Crear materia

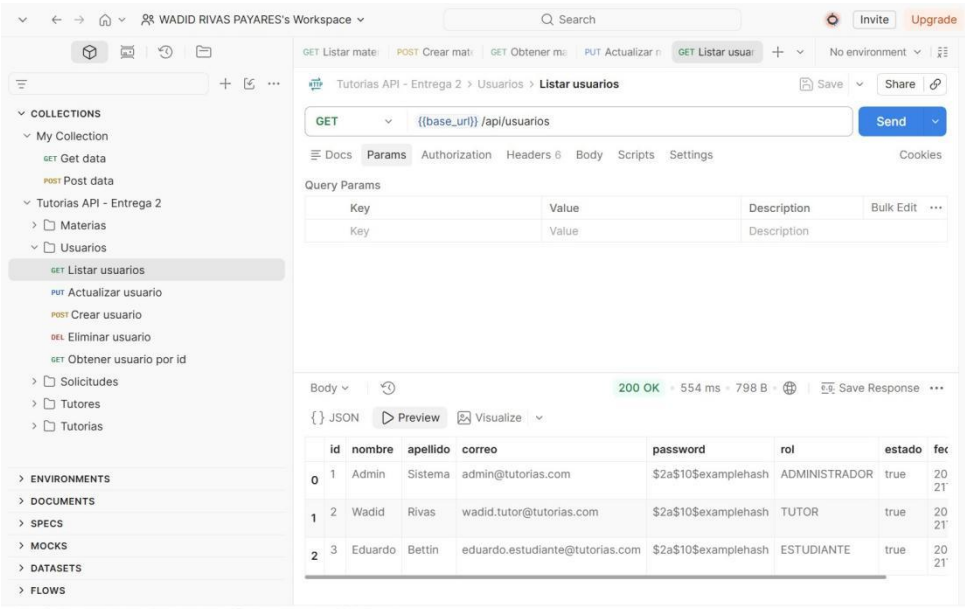


Actualizar Materia



Usuarios

Lista Usuario



11. Repositorio en GitHub

- Repositorio en GitHub:

<https://github.com/WRivxs/SGTU-Tutorias-Universitarias>

11.1 Resumen del contenido del README

El README describe brevemente el proyecto, indicando su objetivo y el stack tecnológico que utiliza: Spring Boot 3, MySQL 8, Lombok y JUnit 5. Incluye un diagrama de capas que muestra la arquitectura de cuatro niveles (configuración, controladores, servicios y repositorios) y un árbol de paquetes que detalla la organización del código fuente. En los requisitos previos, se enumeran JDK 17, Maven 3.x y PowerShell (para Windows) o Bash (para Linux/macOS) como dependencias necesarias. Se indican las variables de entorno que debe proporcionar el usuario (DB_HOST, DB_PORT, DB_NAME, DB_USERNAME, DB_PASSWORD y SERVER_PORT) y se explica que el script `run-local.ps1` configura esas variables y lanza `mvn spring-boot:run`. También se documentan los comandos estándar de compilación y pruebas (`mvn clean install`, `mvn test`), y se ofrece una tabla de referencia rápida con los endpoints REST disponibles. Se menciona la colección de Postman incluida, junto con una breve guía de importación, y los pasos para desplegar la base de datos en AWS RDS ejecutando el script `sql/ddl_tutorias.sql`. Finalmente, se describen las normas de contribución, los lineamientos de commit y el archivo `.gitignore`, que excluye credenciales y el script `run-local.ps1`.

12. Conclusiones y Retos Encontrados

Durante el modelado de datos surgió la dificultad de detectar redundancias entre usuarios y tutores en la versión preliminar; la solución consistió en crear una tabla tutores con una relación 1-0..1, logrando una normalización en tercera forma normal (3FN). La conexión a AWS RDS se complicó porque la VPC del proyecto bloqueaba el puerto 3306; se resolvió configurando el Security Group para permitir tráfico inbound solo desde la IP del desarrollador y utilizando variables de entorno para la URL de la base de datos. En la configuración de

Spring Boot, el archivo `application.yml` generaba errores de parsing cuando faltaban comillas en los valores de los enum; se adoptó la sintaxis `${}` y se verificó localmente con `mvn spring-boot:run` antes de confirmar los cambios. La validación de datos fue otro punto crítico, ya que algunos DTOs carecían de restricciones y permitían información inválida; se añadió `Bean Validation` (`@NotBlank`, `@Email`, `@Size`) y la anotación `@Valid` en los controladores. Las pruebas con Postman mostraron respuestas 500 al intentar crear tutorías con un `tutor_id` inexistente; se implementó una verificación de existencia en el servicio (`tutorRepository.findById`) y se devolvió 404 con un mensaje claro. Por último, mantener sincronizado el README, el script `run-local.ps1` y la colección de Postman resultó engorroso, por lo que se incorporó una sección “Checklist antes de subir” y se automatizó la generación de la colección mediante pruebas de integración.